DOSSIER DE PROJET

DRIVEO

Location de voitures de luxe

Janice Kouakou
Holberton School
Stages 1 à 3 · Mis à jour en novembre 2026



Sommaire

Stage 1 **Équipe, idées et MVP**

Stage 2 **Planification du projet**

Stage 3 **Documentation technique**

Driveo est un site de location de voitures de luxe : catalogue, prix en direct, comptes utilisateurs et réservations sécurisées. Code : github.com/Janice91/metricflow · Site : metricflow-tan.vercel.app

STAGE 1

Équipe, idées et MVP

1. Team Formation Overview

Pour Driveo, je mène le projet en autonomie : je cumule les rôles techniques et la gestion de projet. Mes mentors Holberton et les futurs utilisateurs du site sont les parties prenantes qui cadrent et valident mon travail.

Membre et rôle principal

Membre	Rôle principal	Pourquoi ce rôle
Moi, Janice Kouakou	Project Manager temporaire et Fullstack Developer	J'ai de l'expérience en Python, JavaScript, Flask, MongoDB, Docker et API REST grâce aux projets HBnB, puis en Next.js et Supabase avec MetricFlow. Je peux coder le front et le back et coordonner l'ensemble.

Rôles techniques et responsabilités

Je distingue chaque rôle, même si je les tiens tous, pour que mon travail reste bien structuré et que je sache quelles décisions relèvent de quel rôle.

Rôle	Responsabilités	Outils
Project Manager temporaire	Planifier les étapes, suivre l'avancement, prendre les décisions, faire le lien avec les mentors	Liste de tâches, journal d'avancement
Frontend Developer	Pages (accueil, catalogue, fiche voiture, connexion, réservation, mes réservations), composants, responsive	Next.js 16, React 19, TypeScript, Tailwind CSS
Backend Developer	Comptes utilisateurs, base de données des réservations, règles de sécurité (RLS)	Supabase Auth, Postgres, supabase-js
UI Designer	Identité noir et or, choix des photos, cohérence visuelle	Tailwind CSS, photos Wikimedia Commons
DevOps	Versionner le code, mettre en ligne, gérer les variables d'environnement	Git, GitHub, Vercel CLI
QA / testeur	Tester le parcours complet (recherche, connexion, réservation, annulation) et corriger	Navigateur, tests manuels

Normes de communication et de décision

- **Suivi quotidien** : je commence chaque session par un point de 15 minutes pour fixer la tâche du jour, et je fais un bilan à la fin.
- **Code** : je versionne sur GitHub, avec un commit par fonctionnalité et un message clair.
- **Retours externes** : je pose mes questions aux mentors Holberton et j'intègre leur feedback à chaque étape.
- **Règle de décision** : je choisis selon quatre critères (faisabilité dans le temps imparti, utilité pour l'utilisateur, alignement avec les technologies apprises, évolutivité). En cas de doute, je demande l'avis d'un mentor.

Parties prenantes

Partie prenante	Rôle	Impact sur le projet
Mentors et équipe pédagogique Holberton	Évaluateurs	Valident chaque étape, donnent le feedback : impact fort
Utilisateurs finaux (clients souhaitant louer une voiture de luxe)	Utilisateurs cibles	Définissent les besoins : prix clair, réservation rapide, suivi : impact fort
Agences de location (clients à servir)	Commanditaires fictifs du site	Fixent les règles métier : âge minimum, remises, annulation : impact moyen
Supabase, Vercel, GitHub	Fournisseurs techniques	Hébergement, comptes, base de données, code : impact moyen
Auteurs des photos (Wikimedia Commons)	Fournisseurs d'images libres	Fournissent les photos, créditées sur le site : impact faible
Camarades de promotion	Pairs	Retours et revue du projet : impact faible

2. Ideas Explored (idées brainstormées)

J'ai utilisé le **mind mapping** et les questions **« How Might We »** pour explorer plusieurs pistes autour de problèmes concrets du quotidien et des petites entreprises.

#	Idée	Description courte	Raison du rejet ou du choix
1	App de gestion de tâches collaborative	Un Trello simplifié pour petites équipes	Marché déjà saturé (Trello, Notion, Asana), peu de valeur différenciante
2	App de suivi de dépenses personnelles	Suivi de budget avec visualisation	Trop simple techniquement, peu de défi côté back-end
3	Plateforme de mise en relation freelances	Matching freelances et clients	Scope trop large : modération, paiement, aspects légaux
4	App de réservation de salles de sport	Réservation de créneaux	Trop proche d'exemples déjà vus en formation
5	MetricFlow : dashboard pour agences de publicité	Suivi des performances publicitaires et rapports IA	Bonne idée, mais dépend d'API externes (Meta Ads) difficiles à tester et peu visuelle à présenter. J'en garde la base technique (Next.js, Supabase).
6	**Driveo : site de location de voitures de luxe**	Choisir une voiture, voir le prix total, réserver et suivre ses locations	**Retenue**

Critères d'évaluation utilisés

- **Faisabilité technique** dans le temps imparti et avec mes compétences actuelles.
- **Utilité réelle** pour un utilisateur concret.
- **Alignement avec les technologies apprises** (Next.js, API REST, bases de données).
- **Évolutivité** : le projet peut grandir sur les étapes suivantes.
- **Impact visuel** : un site que l'on peut montrer et démontrer en direct.

3. Selected MVP Concept — Driveo

Résumé

Driveo est un site web de **location de voitures de luxe**. Un client peut parcourir un catalogue de 12 voitures (Urus Performante, Corvette C8, Ferrari 296 GTB, Rolls-Royce Cullinan…), choisir ses dates et ses options, voir **le prix total en direct**, créer un compte, réserver et retrouver ses réservations dans « Mes réservations ».

Le problème qu'il résout

- Les prix sont souvent flous et des frais apparaissent à la fin.
- Réserver prend du temps : appels, e-mails, longs formulaires.
- Il n'y a aucun endroit simple pour retrouver ou annuler sa réservation.

La solution proposée

- Un **catalogue filtrable** (catégorie, carburant, boîte, places, prix) avec photos.
- Un **prix calculé en direct** : remises automatiques de 5 %, 10 % et 15 % selon la durée, 5 options à la journée.
- Des **comptes utilisateurs** sécurisés (Supabase Auth) et des réservations enregistrées en base de données, protégées par une règle de sécurité (RLS).

Public cible

Des clients adultes (25 ans minimum) souhaitant louer une voiture de luxe pour un week-end, un mariage ou un événement.

Type d'application

Application **web** responsive, accessible depuis un navigateur. Pas d'application mobile pour ce MVP.

Pourquoi cette idée plutôt que les autres

- Elle est **plus originale et plus visuelle** que les idées 1, 2 et 4.
- Elle est **plus réaliste dans le temps imparti** que la plateforme de freelances (idée 3) : pas de paiement ni de modération.
- Elle couvre un **périmètre fullstack complet** : pages, calcul de prix, authentification, base de données, sécurité, déploiement.
- Elle reprend l'expérience de MetricFlow (Next.js, Supabase) sans dépendre d'une API externe fragile.
- Elle est **facile à démontrer en direct** devant un public.

Objectifs clés (SMART)

1. **Catalogue et prix** : afficher 12 voitures avec photos, filtres et prix total recalculé selon les dates et les options, livré au Stage 4.
2. **Comptes et réservations** : permettre de créer un compte, de réserver et d'annuler une réservation, chaque utilisateur ne voyant que les siennes, livré au Stage 4.
3. **Mise en ligne** : site accessible publiquement sur Vercel avec un parcours complet testé avant la présentation finale.

Scope (in / out)

In-scope :

- Catalogue, filtres, fiche voiture, 6 agences
- Calcul du prix avec options et remises
- Inscription, connexion, déconnexion
- Réservation, liste « Mes réservations », annulation

Out-of-scope :

- Paiement en ligne
- E-mail de confirmation réel
- Gestion des disponibilités (éviter la double réservation)
- Application mobile native

Risques et défis identifiés

Risque	Mitigation
Sécurité des données de réservation	Utiliser Supabase Auth et des règles RLS pour que chacun ne voie que ses réservations
Photos de voitures : droits d'utilisation	Utiliser des photos sous licence libre (Wikimedia Commons) et les créditer
Dépendance à un service gratuit (Supabase peut se mettre en pause)	Vérifier le projet avant les démos et savoir le restaurer
Environnement de travail qui se réinitialise	Tout faire en ligne de commande et garder le code sur GitHub
Temps limité	Prioriser les fonctions In-scope, garder le paiement pour plus tard

Documentation — Synthèse du processus

Équipe : je suis seul(e) sur ce projet ; j'ai défini les rôles (PM, front, back, design, DevOps, QA) pour rester organisé, avec les mentors Holberton comme repère.

Brainstorming : 6 idées explorées, évaluées selon la faisabilité, l'utilité, l'alignement technique, l'évolutivité et l'impact visuel.

MVP retenu : Driveo, un site de location de voitures de luxe avec prix en direct, comptes utilisateurs et réservations sécurisées.

STAGE 2

Planification du projet

Project Charter — High-Level Plan

Driveo est un site de location de voitures de luxe : le client choisit une voiture, voit le prix total en direct, crée un compte et réserve en quelques clics. Ce document présente le plan global du projet, phase par phase, du lancement jusqu'à la clôture.

Objectifs du projet

- Livrer un site complet et en ligne, avec un parcours client de bout en bout (découvrir, choisir, personnaliser, se connecter, réserver et suivre).
- Garantir la sécurité des données : chaque utilisateur ne voit que ses réservations.
- Pouvoir présenter et démontrer le site en direct.

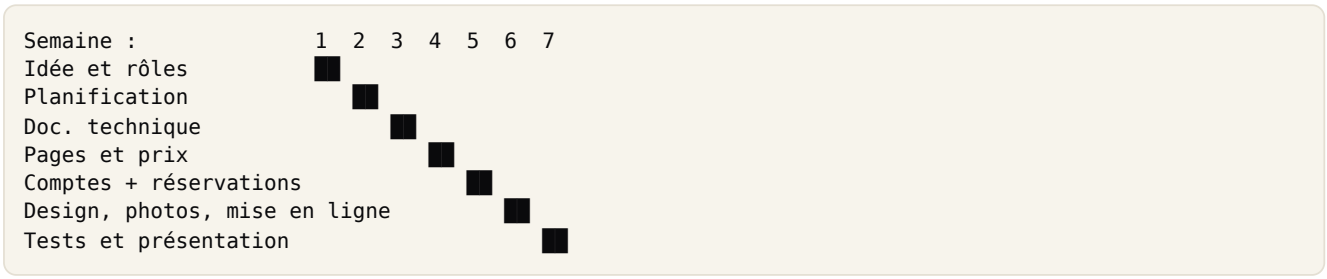
Timeline / Phases du projet

Phase	Période	Objectif	Livrable principal
Stage 1 : Idea Development	Semaine 1	Définir mon rôle, brainstormer, choisir le MVP	Stage 1 Report
Stage 2 : Project Planning	Semaine 2	Définir le plan global du projet	Ce document (Project Charter)
Stage 3 : Technical Documentation	Semaine 3	Concevoir l'architecture, la base de données, les user stories	Documentation technique
Stage 4 : MVP Development	Semaines 4 à 6	Développer les fonctionnalités du MVP	Site fonctionnel en ligne
Stage 5 : Project Closure	Semaine 7	Tester, corriger, préparer la présentation	Présentation finale et démo

Jalons clés

- **Semaine 1** : rôles définis, idée Driveo validée.
- **Semaine 2** : plan global validé, priorités fixées.
- **Semaine 3** : user stories priorisées (MoSCoW), architecture, schéma de base de données.
- **Semaine 4** : pages principales (accueil, catalogue, fiche voiture) et calcul des prix.
- **Semaine 5** : comptes utilisateurs (Supabase Auth) et réservations en base de données avec RLS.
- **Semaine 6** : design noir et or, photos de voitures, mise en ligne sur Vercel.
- **Semaine 7** : tests du parcours complet, corrections, présentation PowerPoint et démo.

Répartition simplifiée (Gantt textuel)



Ressources et outils

Besoin	Outil
Code	Next.js, React, TypeScript, Tailwind CSS
Comptes et base de données	Supabase (Auth, Postgres, RLS)
Versionnement	Git et GitHub
Mise en ligne	Vercel CLI
Photos	Wikimedia Commons (licences libres)

Risques de planification

Risque	Plan B
Retard sur une phase	Réduire le scope (fonctions « Could Have ») plutôt que le délai final
Projet Supabase gratuit en pause	Le restaurer depuis le tableau de bord avant chaque démo
Environnement de travail réinitialisé	Garder tout le code sur GitHub et refaire l'installation en ligne de commande

Points de vigilance pour la suite

- Faire un point de 15 minutes en début de chaque session et un bilan hebdomadaire.
- Terminer le Stage 3 (documentation technique) avant de commencer le développement, pour éviter les allers-retours.
- Tester sur le vrai site en ligne, pas seulement en local.

STAGE 3

Documentation technique

1. User Stories and Mockups

User Stories (priorisées avec MoSCoW)

Chaque story a des critères d'acceptation (« Étant donné / Quand / Alors ») qui servent ensuite de base aux cas de test de la section 6.

Clients

ID	User story	Priorité	Critères d'acceptation	Statut
US-01	En tant que client, je veux parcourir un catalogue avec photos et prix, afin de choisir une voiture.	Must	Quand j'ouvre `/voitures`, alors je vois toutes les voitures visibles avec photo, prix par jour et note.	Fait
US-02	En tant que client, je veux filtrer et trier (catégorie, carburant, boîte, places, prix max).	Must	Quand je choisis un filtre, alors la liste se met à jour sans recharger la page ; « Réinitialiser » remet tout à zéro.	Fait
US-03	En tant que client, je veux choisir agence, dates et options et voir le prix total en direct.	Must	Quand je change les dates ou une option, alors le total et la remise longue durée (5 %, 10 %, 15 %) se recalculent.	Fait
US-04	En tant que visiteur, je dois créer un compte avant de réserver.	Must	Étant donné que je ne suis pas connecté, quand je clique sur « Réserver », alors je suis envoyé à l'inscription puis ramené à ma réservation avec voiture et dates conservées. Le serveur refuse aussi tout paiement sans compte (HTTP 401).	Fait
US-05	En tant que client, je veux payer en ligne par carte de façon sécurisée.	Must	Quand je clique sur « Payer », alors je suis redirigé vers Stripe Checkout (mode test) ; après paiement, ma réservation est enregistrée et un numéro m'est affiché.	Fait (mode test)
US-06	En tant que client, je veux retrouver mes réservations.	Must	Étant donné que je suis connecté, alors « Mes réservations » liste uniquement les miennes.	Fait
US-07	En tant que client, je veux annuler une réservation.	Should	Quand je clique sur « Annuler » et que je confirme, alors elle disparaît de ma liste.	Fait
US-08	En tant que client, je veux que mes coordonnées soient pré-remplies.	Should	Quand j'ouvre la page de réservation, alors nom et e-mail viennent de mon compte.	Fait
US-09	En tant que client, je veux voir les disponibilités réelles d'une voiture.	Could	Une voiture déjà réservée aux mêmes dates est signalée indisponible.	À faire
US-10	En tant que client, je veux recevoir un e-mail de confirmation.	Could	Un e-mail part après paiement confirmé.	À faire

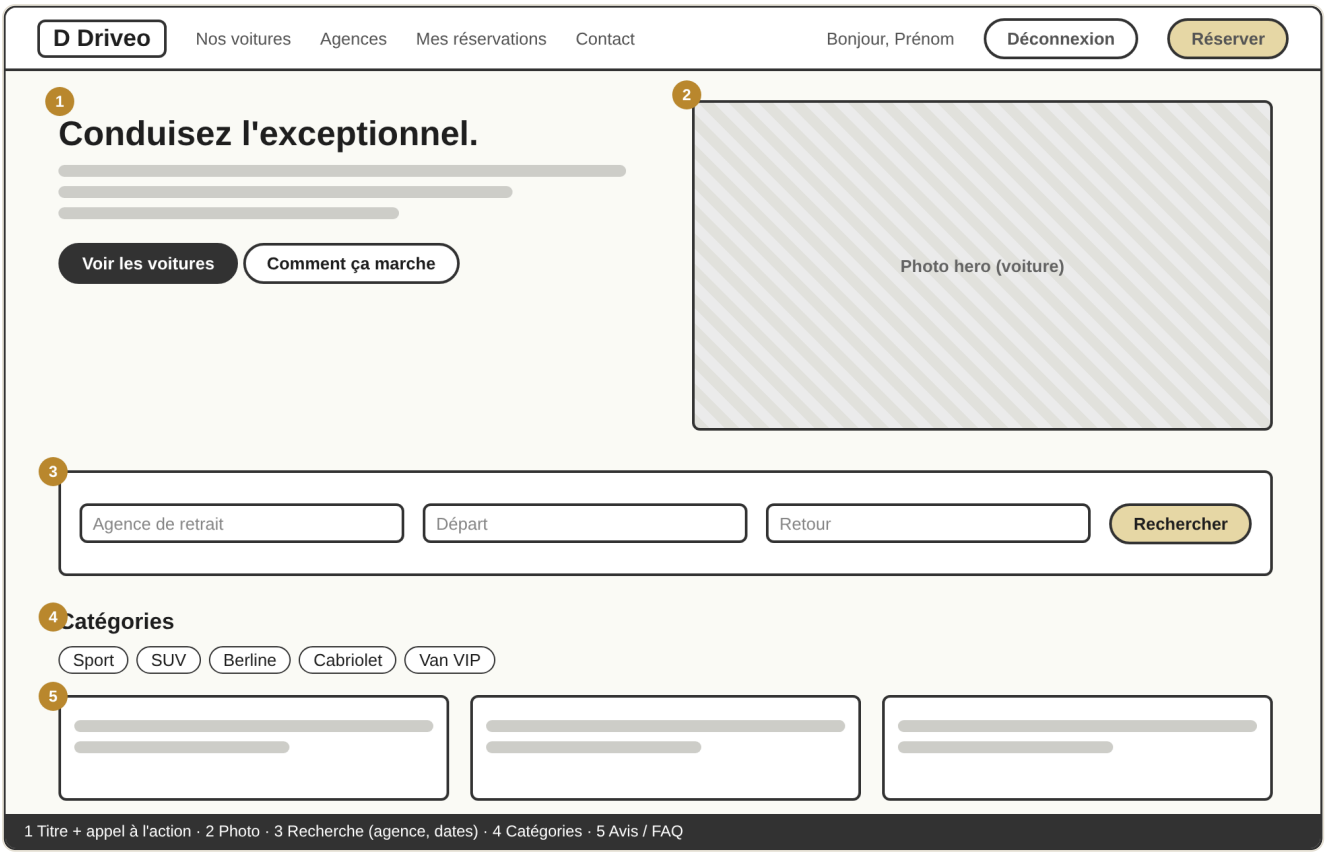
Administrateur

ID	User story	Priorité	Critères d'acceptation	Statut
US-11	En tant qu'admin, je veux une page d'administration réservée aux admins.	Must	Un compte non admin qui ouvre `/admin` voit « Accès refusé » ; un admin y arrive directement après connexion.	Fait
US-12	En tant qu'admin, je veux voir et annuler toutes les réservations, avec recherche et chiffres clés.	Must	Le tableau liste toutes les réservations de tous les clients ; recherche par client, e-mail, voiture ou numéro.	Fait
US-13	En tant qu'admin, je veux ajouter, modifier, masquer et supprimer des voitures.	Must	Une voiture masquée disparaît du site public mais reste dans l'admin ; un changement de prix est appliqué au prochain paiement.	Fait
US-14	En tant qu'admin, je veux gérer les agences (mêmes actions).	Should	Idem voitures ; une agence masquée n'apparaît plus dans les listes de retrait.	Fait
US-15	En tant qu'admin, je veux consulter la liste des clients inscrits.	Should	Liste des comptes avec date d'inscription, dernière connexion, nombre de réservations et total dépensé.	Fait

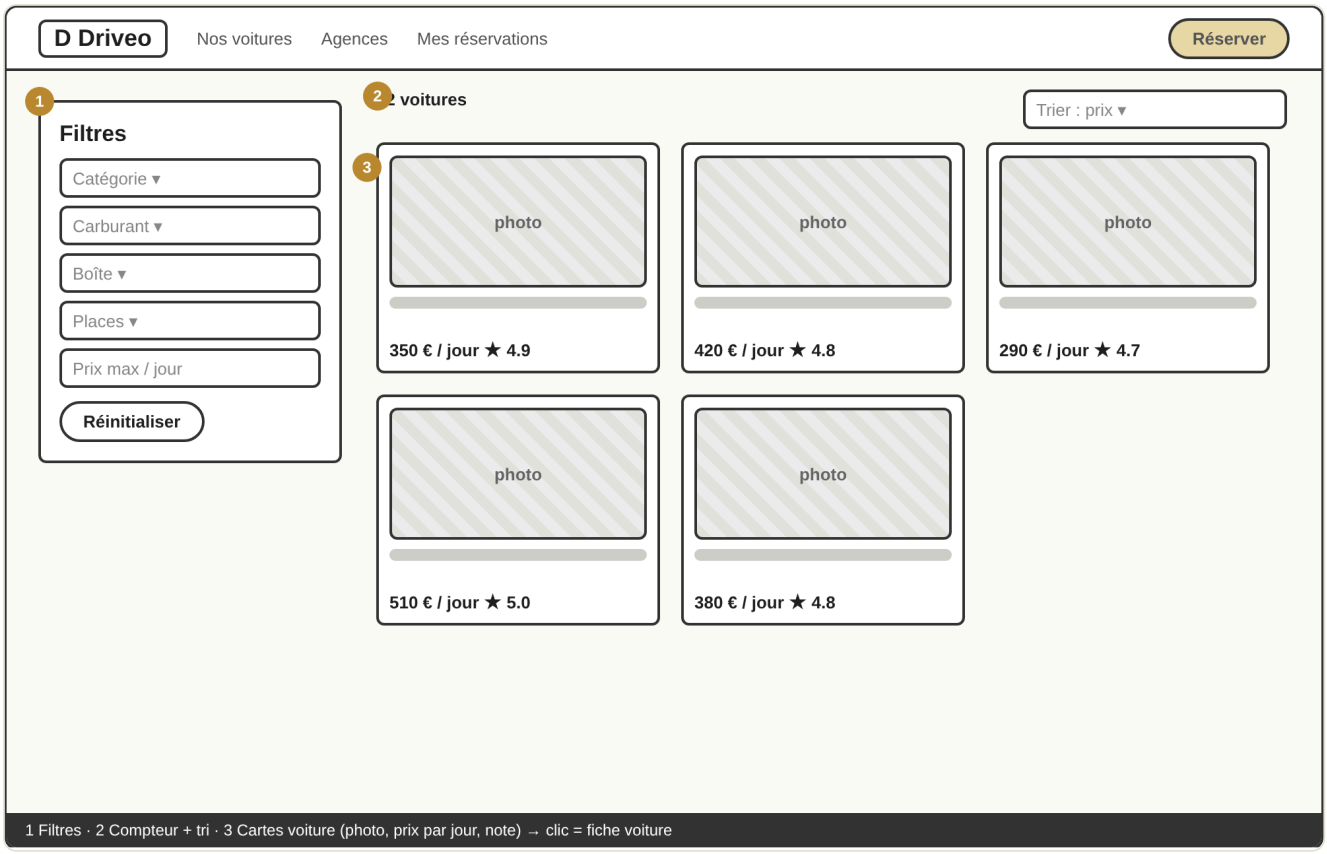
Won't Have (hors scope de cette version) : paiement en mode réel, remboursement automatique, webhook Stripe, application mobile native, multilingue.

Mockups (maquettes front-end)

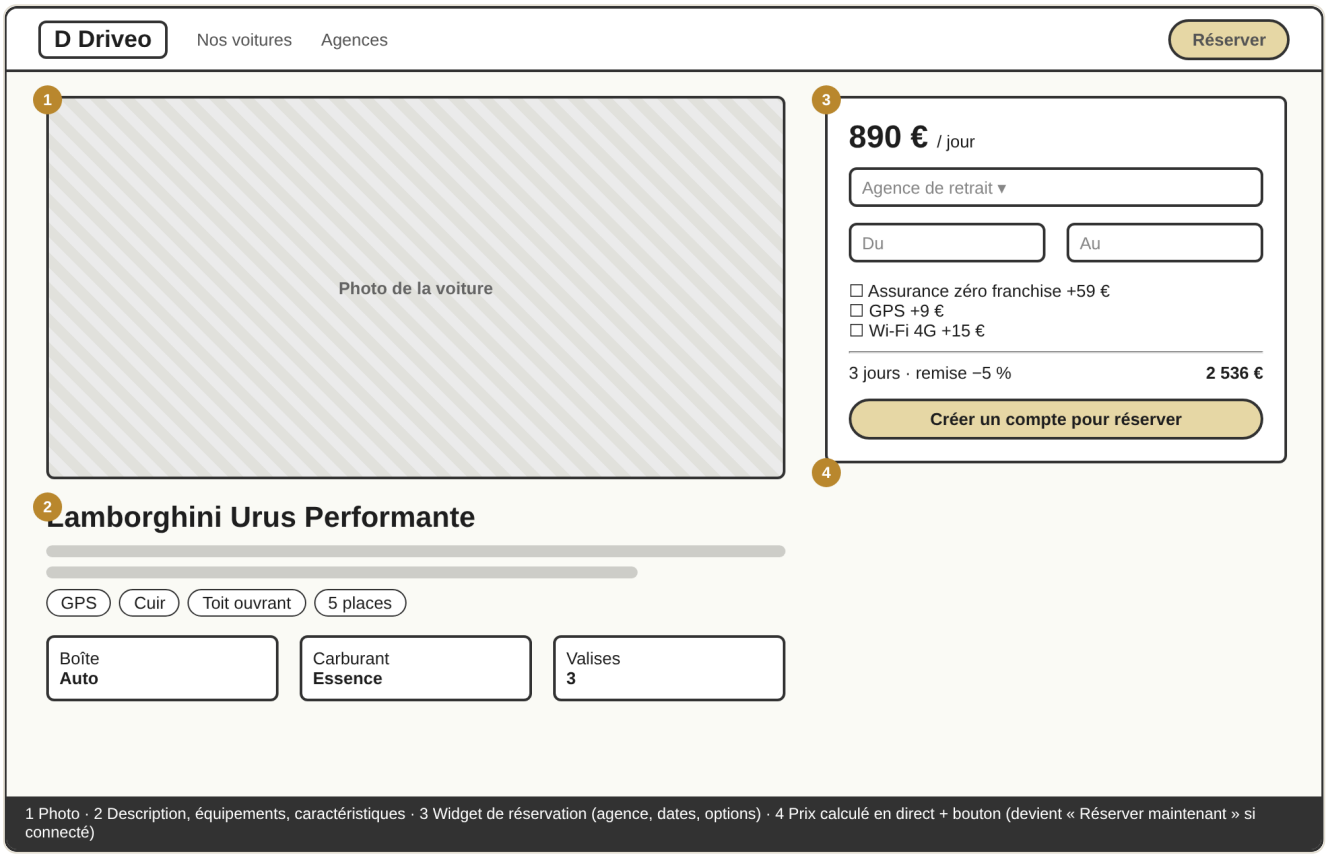
Les maquettes ci-dessous sont des wireframes des écrans principaux, avec des repères numérotés expliqués dans la légende de chaque écran. Ils couvrent le parcours client (écrans 1 à 6), l'administration (écrans 7 et 8) et la version mobile. Ils sont suivis des captures du site réellement développé, pour comparer maquette et résultat.



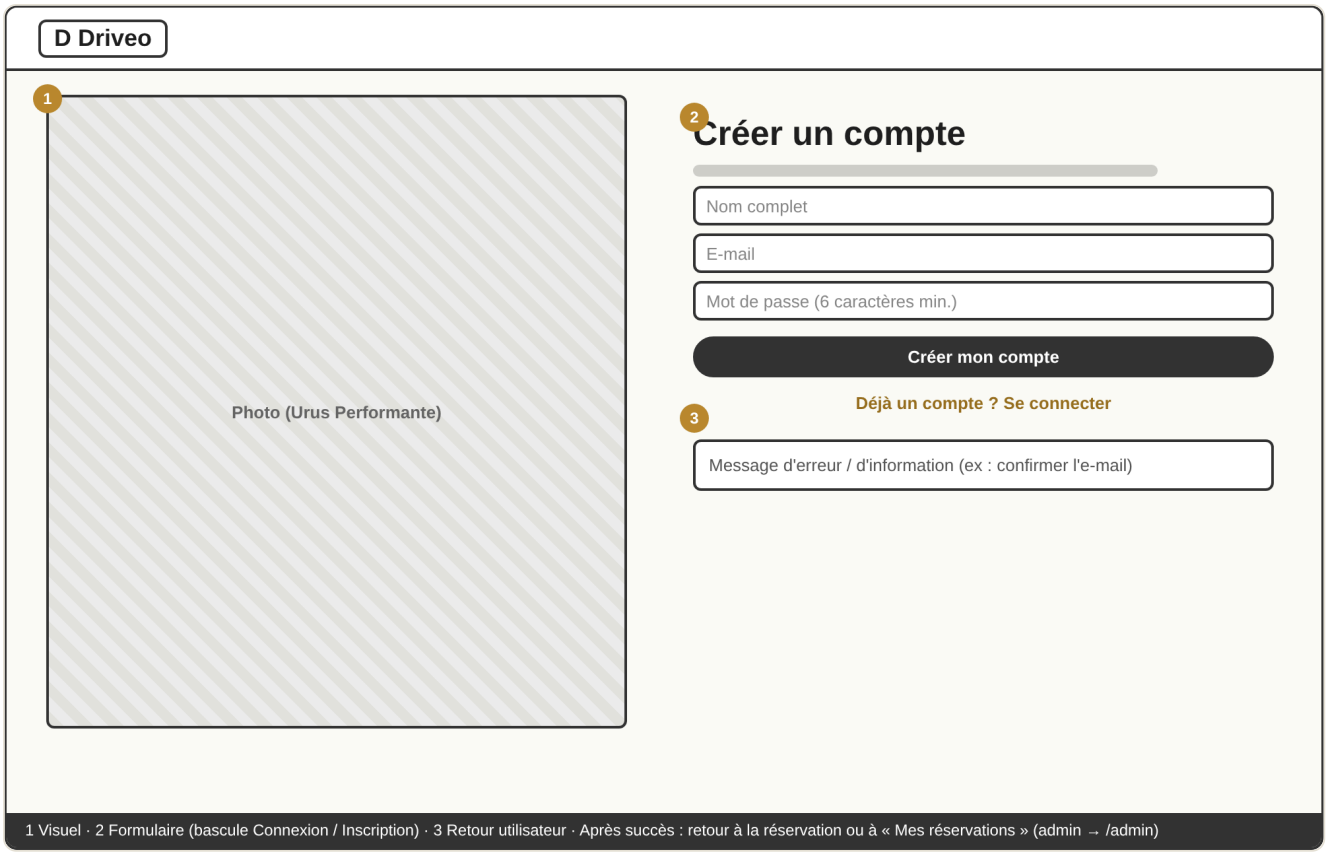
Écran 1 — Accueil



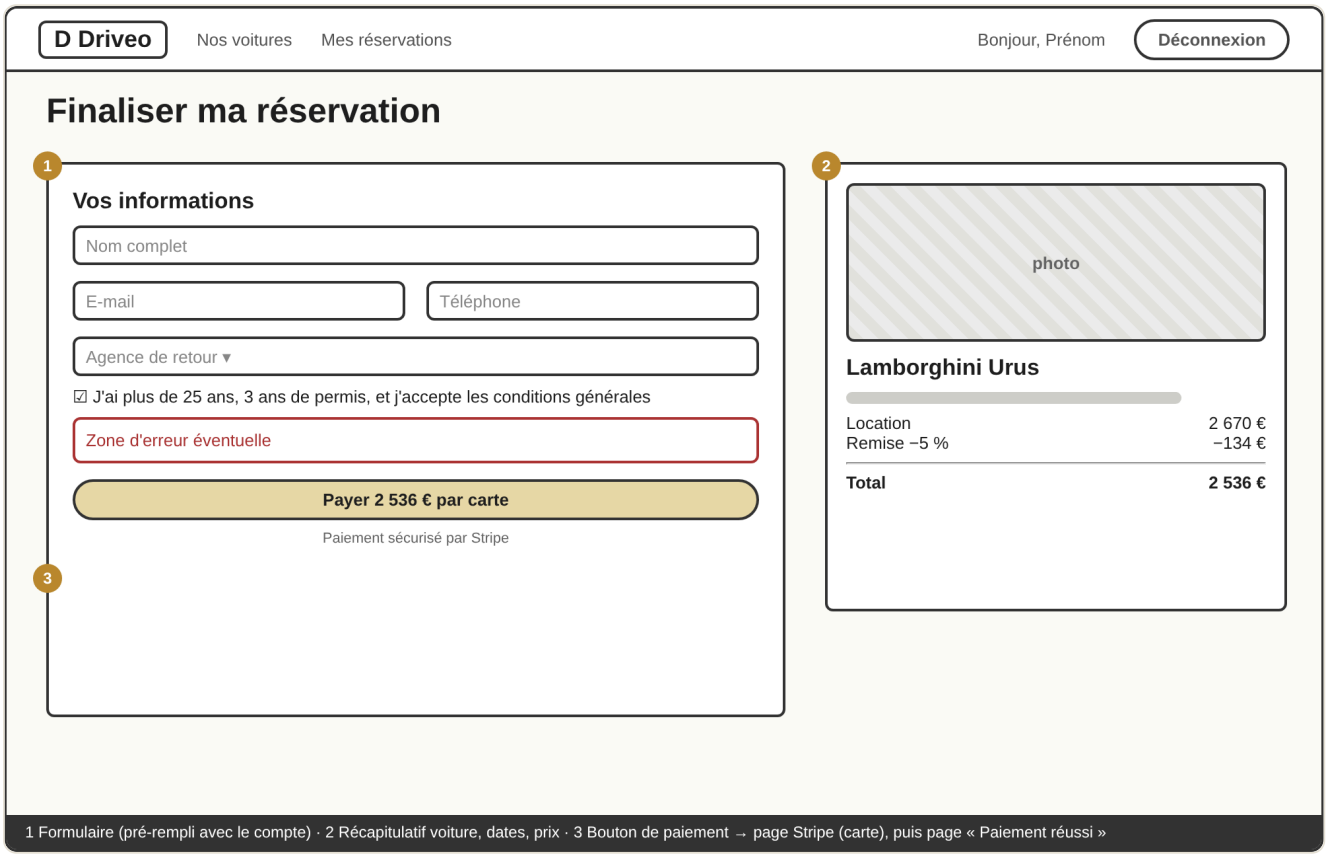
Écran 2 — Catalogue avec filtres



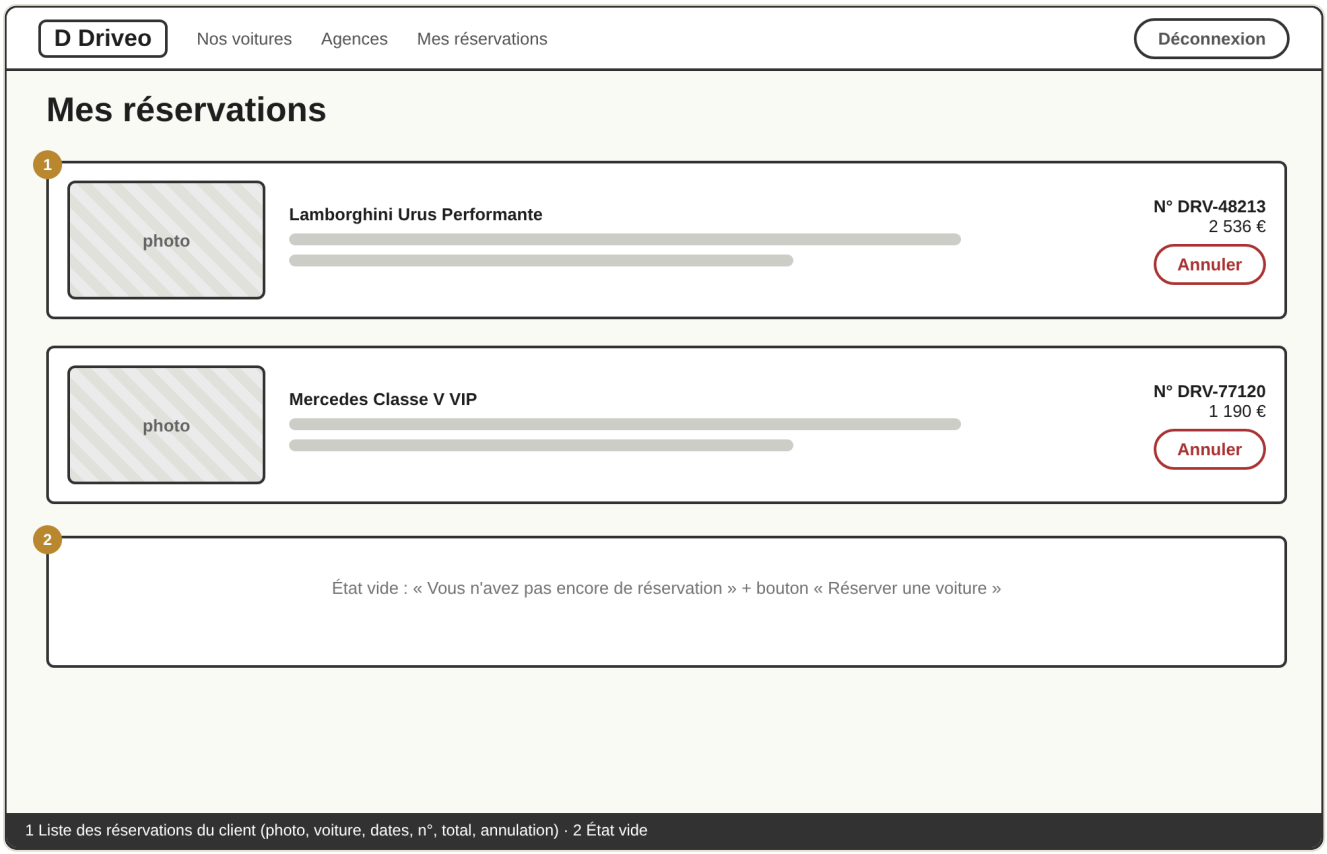
Écran 3 — Fiche voiture et widget de réservation



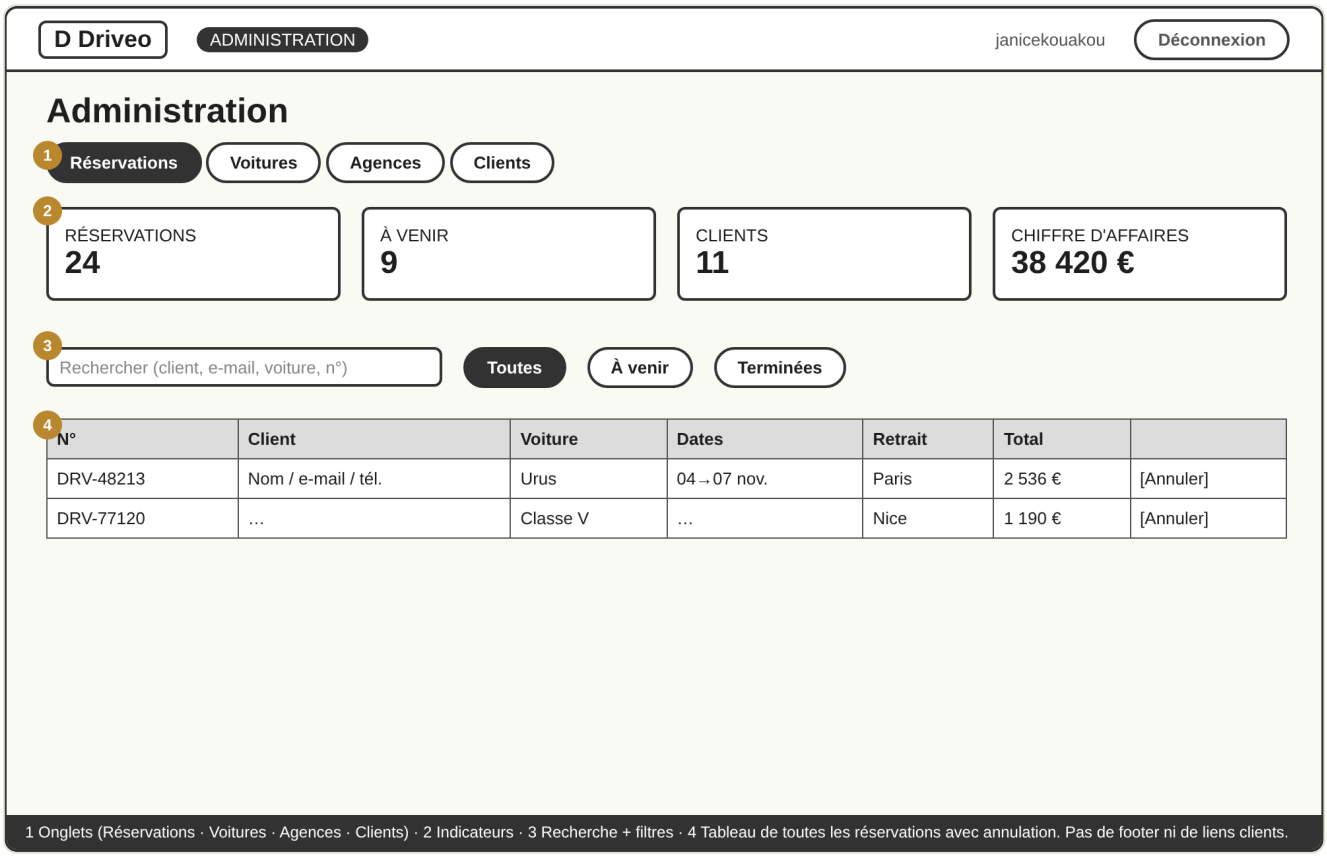
Écran 4 — Inscription / connexion



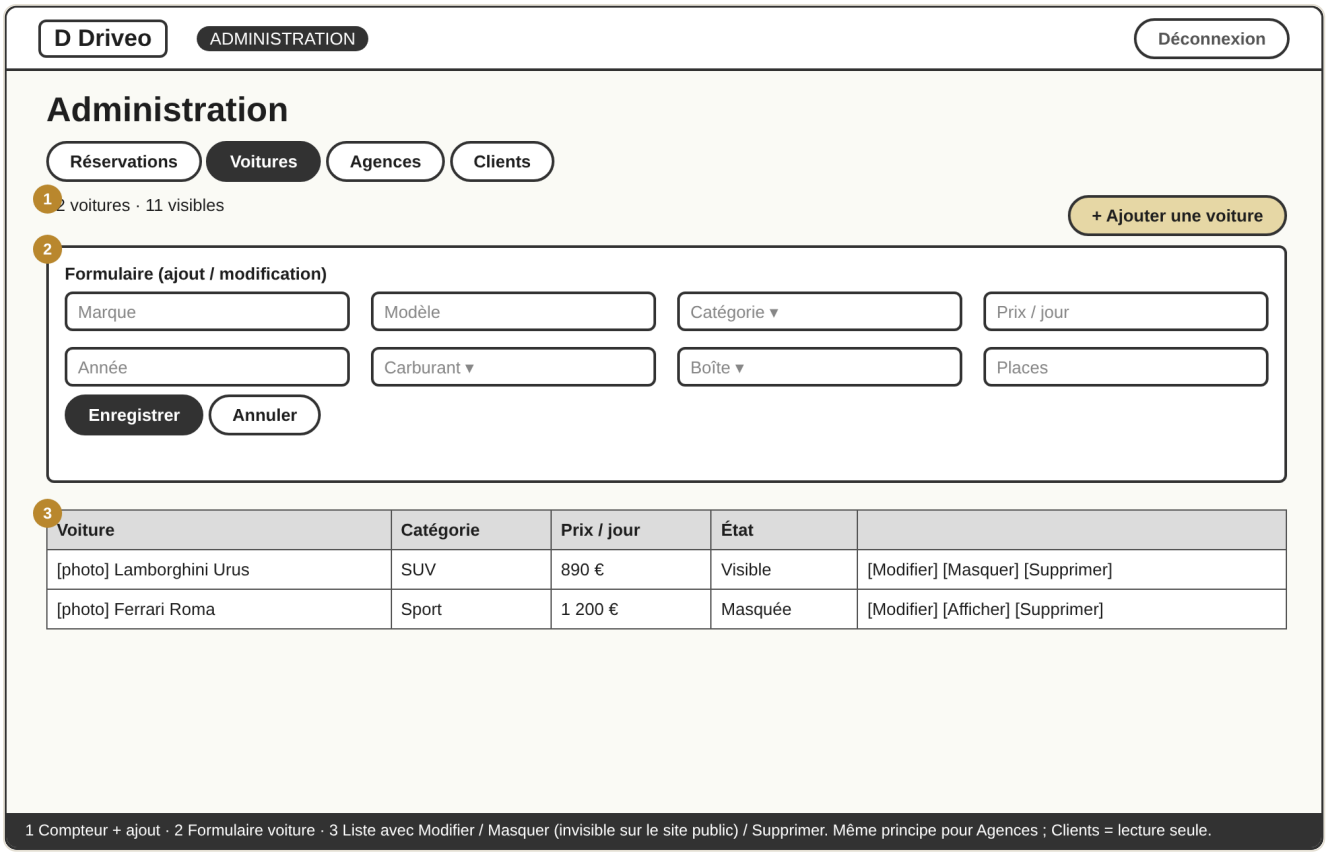
Écran 5 — Finaliser la réservation (paiement)



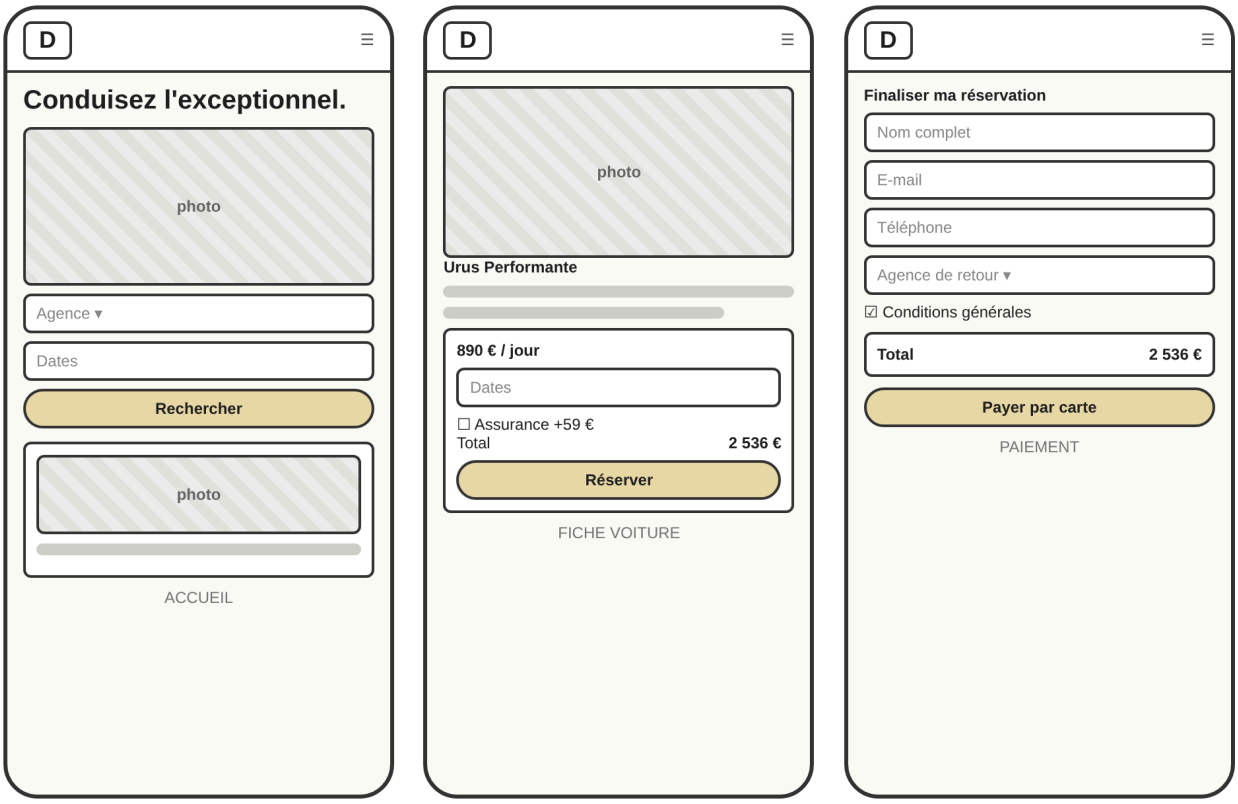
Écran 6 — Mes réservations



Écran 7 — Administration : réservations

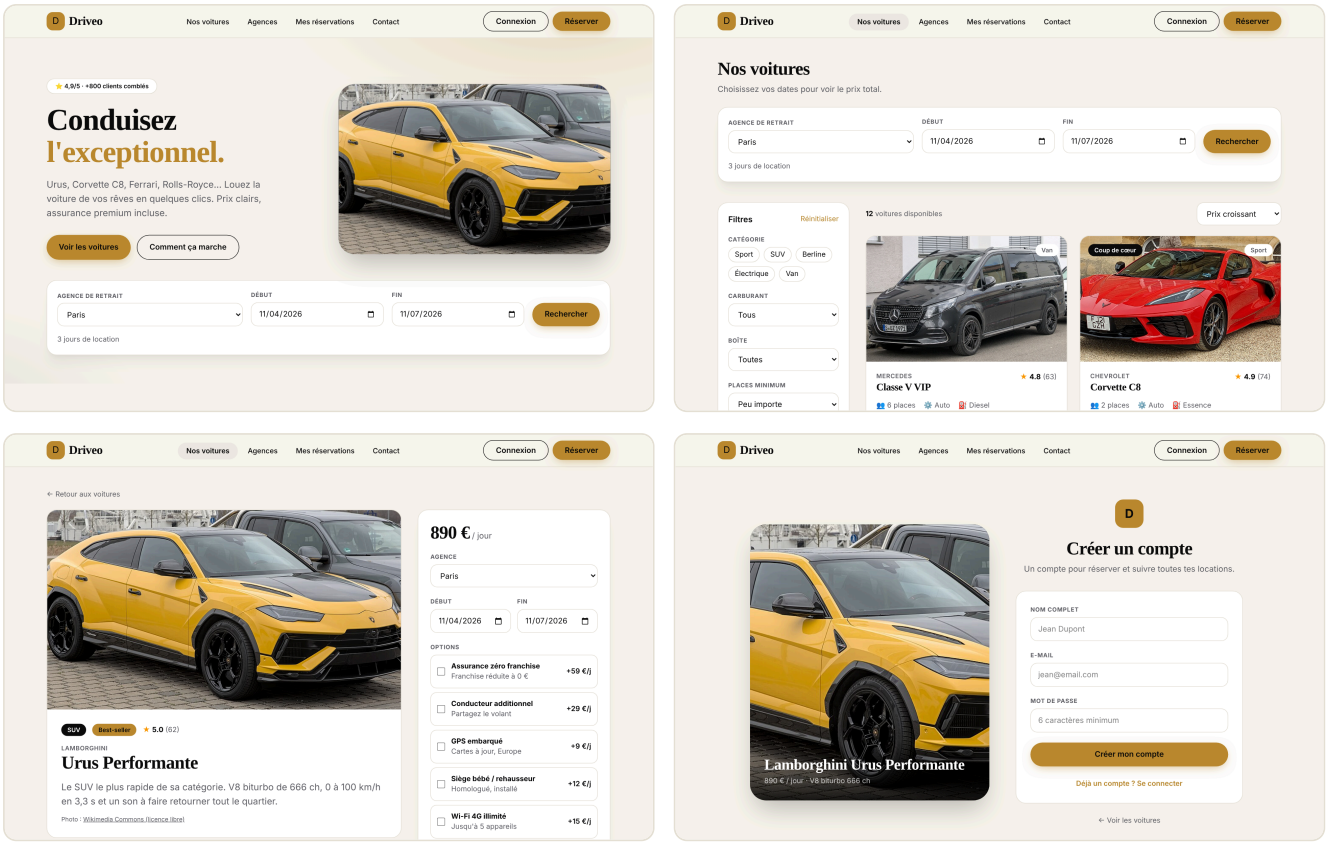


Écran 8 — Administration : voitures (agences et clients suivent le même schéma)



Version mobile (accueil, fiche voiture, paiement) — mise en page en une colonne

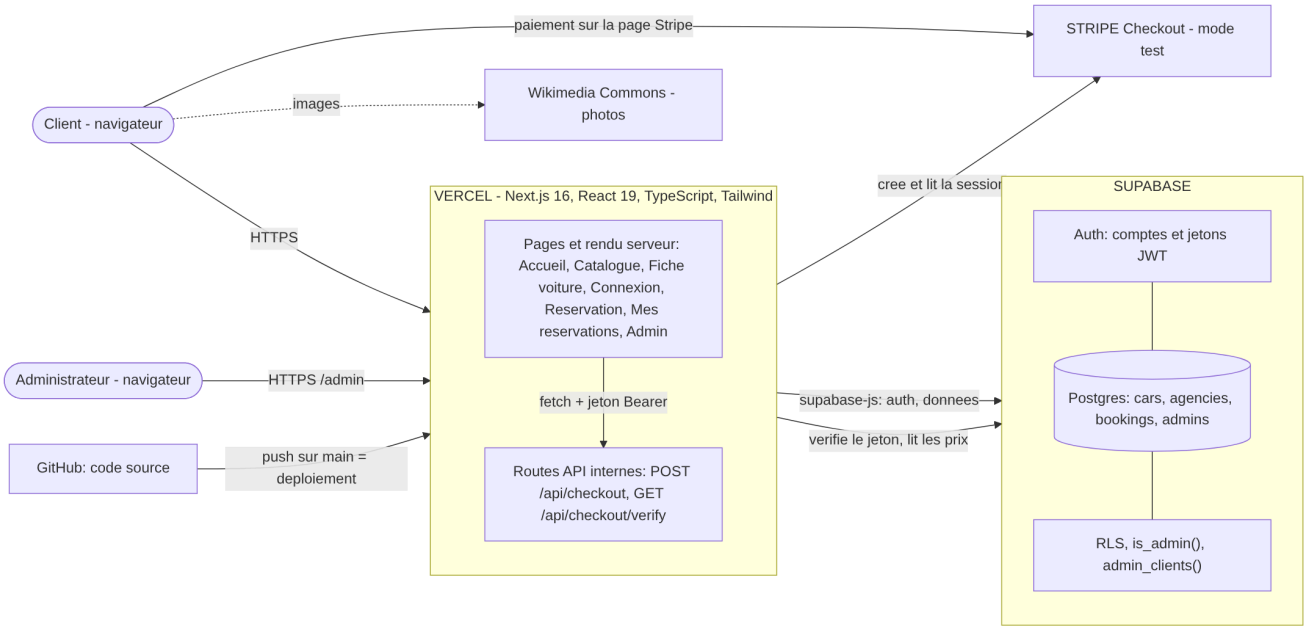
Rendu final (captures du site développé)



Choix de design : thème noir et or pour l'idée de luxe, grandes photos libres de droits, un seul appel à l'action par écran, formulaires courts, prix toujours visible. Sur mobile, les colonnes passent en une seule colonne et le menu devient un bouton.

---

2. System Architecture



Flux de données

1. Le navigateur charge les pages Next.js servies par Vercel. Le serveur lit les voitures et agences dans Supabase (lecture publique) et les passe aux composants via un contexte React (`CatalogProvider`). Si les

tables sont vides ou injoignables, le site retombe sur les données du code : il ne tombe jamais en panne.

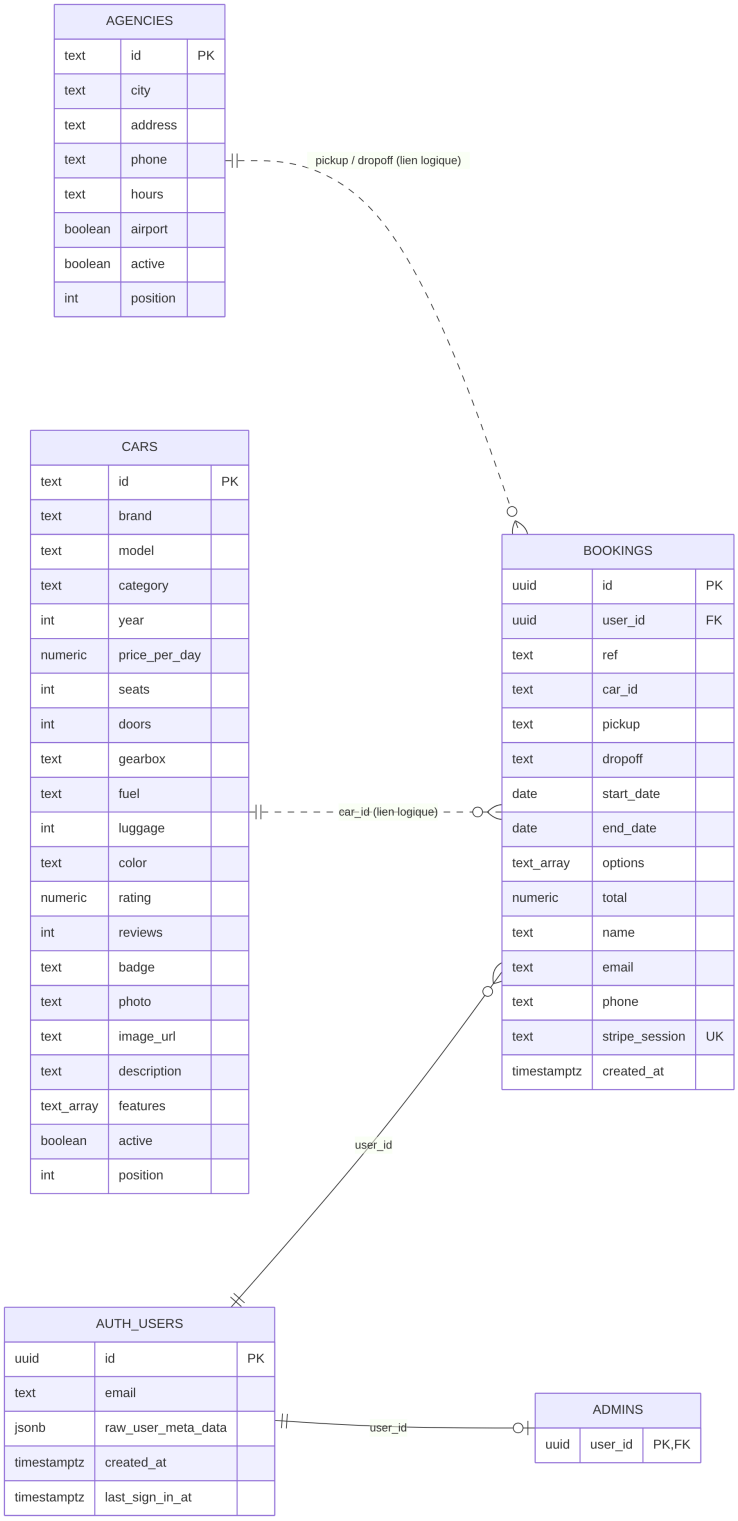
2. Pour s'inscrire ou se connecter, le front appelle Supabase Auth avec supabase-js ; Supabase renvoie un jeton JWT gardé dans le navigateur.
3. Pour payer, le front envoie la demande à notre route `POST /api/checkout` avec le jeton. Le serveur vérifie le jeton, recalcule le prix à partir de la base, crée une session Stripe et renvoie son URL.
4. Le client paie sur la page hébergée par Stripe puis revient sur `/reservation/succes`. Le front demande `GET /api/checkout/verify` ; seulement si Stripe confirme le paiement, la réservation est enregistrée dans `bookings` avec l'identifiant de session Stripe.
5. La sécurité des données est assurée par la base (RLS) : un client ne lit et ne supprime que ses réservations, seul un admin modifie le catalogue.

Déploiement et environnements

Environnement	Où	Données	Paiement
Développement	Sandbox Holberton, `npm run dev -- -p 80`	Projet Supabase unique	Stripe mode test
Preview	Vercel, une URL par branche	Idem	Stripe mode test (la variable `STRIPE_SECRET_KEY` doit aussi être ajoutée à cet environnement)
Production	`metricflow-tan.vercel.app`	Idem	Stripe mode test

3. Components, Classes, and Database Design

Diagramme de la base de données (ERD)



Cardinalités : un utilisateur a 0 à n réservations ; un utilisateur est admin si une ligne existe dans `admins` (0 ou 1). Une voiture et une agence apparaissent dans 0 à n réservations.

Choix important : `car_id`, `pickup` et `dropoff` sont des liens logiques et non des clés étrangères. Ainsi, supprimer une voiture ou une agence ne supprime pas l'historique des réservations déjà payées.

Dictionnaire de données

Table `bookings` (réservations payées)

Champ	Type	Contraintes	Description
id	uuid	PK, défaut `gen_random_uuid()`	Identifiant technique
user_id	uuid	FK vers `auth.users`, défaut `auth.uid()`	Propriétaire, rempli par la base
ref	text	unique en pratique	Numéro affiché au client
car_id	text	—	Identifiant de la voiture
pickup, dropoff	text	—	Agences de retrait et de retour
start_date, end_date	date	—	Dates de location
options	text[]	—	Options choisies
total	numeric	—	Prix total payé en euros
name, email, phone	text	—	Coordonnées du client
stripe_session	text	index unique (si non nul)	Session Stripe, empêche d'enregistrer deux fois le même paiement
created_at	timestamptz	défaut `now()`	Date de création

Table `cars` : catalogue géré depuis l'administration. `position` donne l'ordre d'affichage, `active` permet de masquer sans supprimer, `image_url` permet une photo ajoutée par l'admin (sinon `photo` ou un dessin de secours).

Table `agencies` : points de retrait, mêmes principes (`position`, `active`).

Table `admins` : liste des administrateurs (un `user_id` par ligne). Donner ou retirer le rôle = ajouter ou supprimer une ligne.

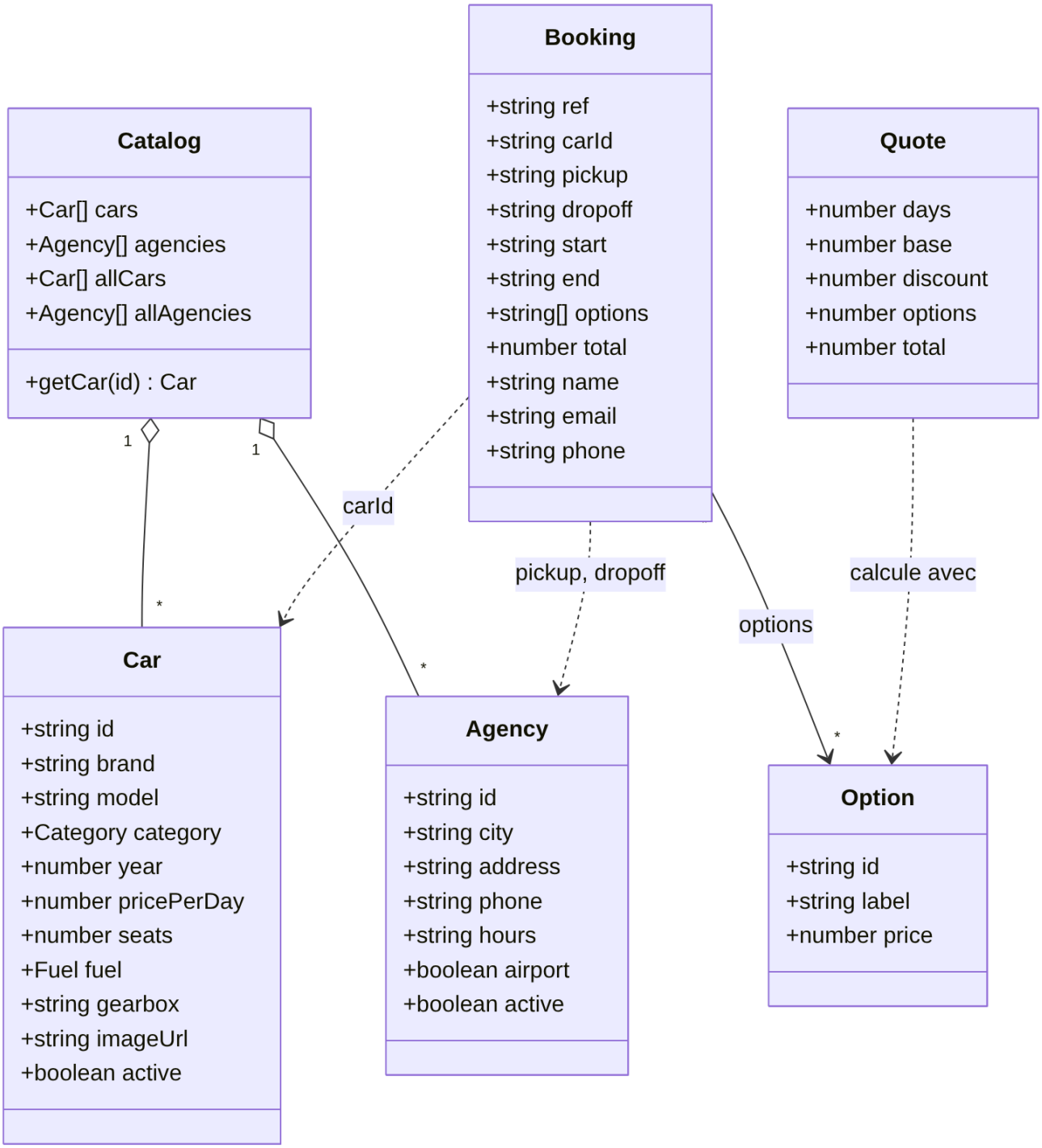
Règles de sécurité (Row Level Security)

Table	Lecture	Écriture
`bookings`	Le propriétaire (`auth.uid() = user_id`) ou un admin	Insertion : propriétaire. Suppression : propriétaire ou admin
`cars`, `agencies`	Tout le monde (catalogue public)	Admin uniquement (`is_admin()`)
`admins`	Chaque utilisateur lit sa propre ligne	Aucune depuis le site (gérée en SQL)

`is_admin()` est une fonction SQL en `security definer` qui vérifie si `auth.uid()` est dans `admins`. La fonction `admin_clients()` (également `security definer`) lit `auth.users` pour la liste des clients et lève une erreur si l'appelant n'est pas admin.

Diagramme de classes (modèle du domaine en TypeScript)

Le projet n'utilise pas de programmation orientée objet : le domaine est décrit par des types TypeScript et des fonctions. Le diagramme montre ces types et leurs relations.


Booking
+string ref
+string carId
+string pickup
+string dropoff
+string start
+string end
+string[] options
+number total
+string name
+string email
+string phone
Catalog
+Car[] cars
+Agency[] agencies
+Car[] allCars
+Agency[] allAgencies
+getCar(id) : Car
Quote
+number days
+number base
+number discount
+number options
+number total
1
1
carId
pickup, dropoff
options
calcule avec
*
*
*
Car
+string id
+string brand
+string model
+Category category
+number year
+number pricePerDay
+number seats
+Fuel fuel
+string gearbox
+string imageUrl
+boolean active
Agency
+string id
+string city
+string address
+string phone
+string hours
+boolean airport
+boolean active
Option
+string id
+string label
+number price

Composants front-end

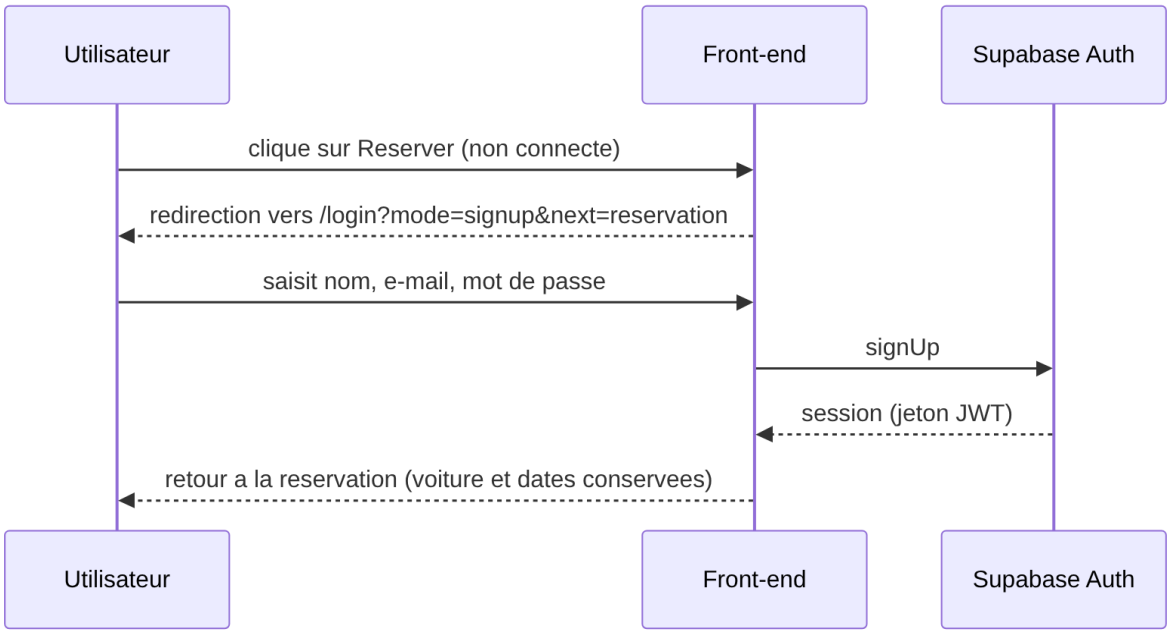
Composant	Rôle
`Navbar`, `Footer`	Navigation ; sur `/admin`, la barre est remplacée par un bandeau « Administration » et le pied de page est masqué
`CarCard`, `CarPhoto`	Carte de voiture ; photo avec repli sur un dessin si l'image manque
`Catalogue`, `SearchForm`	Filtres, tri et recherche par agence et dates
`BookingWidget`	Dates, options, prix en direct ; envoie vers l'inscription si non connecté
`Checkout`	Formulaire de réservation, appel à `/api/checkout`, redirection vers Stripe
`Succes` (`/reservation/succes`)	Vérifie le paiement et enregistre la réservation
`LoginPage`	Inscription et connexion, redirection selon le rôle
`MesReservations`	Liste et annulation
`Admin` + `ReservationsTab`, `VoituresTab`, `AgencesTab`, `ClientsTab`	Back-office à onglets

Fichiers de logique

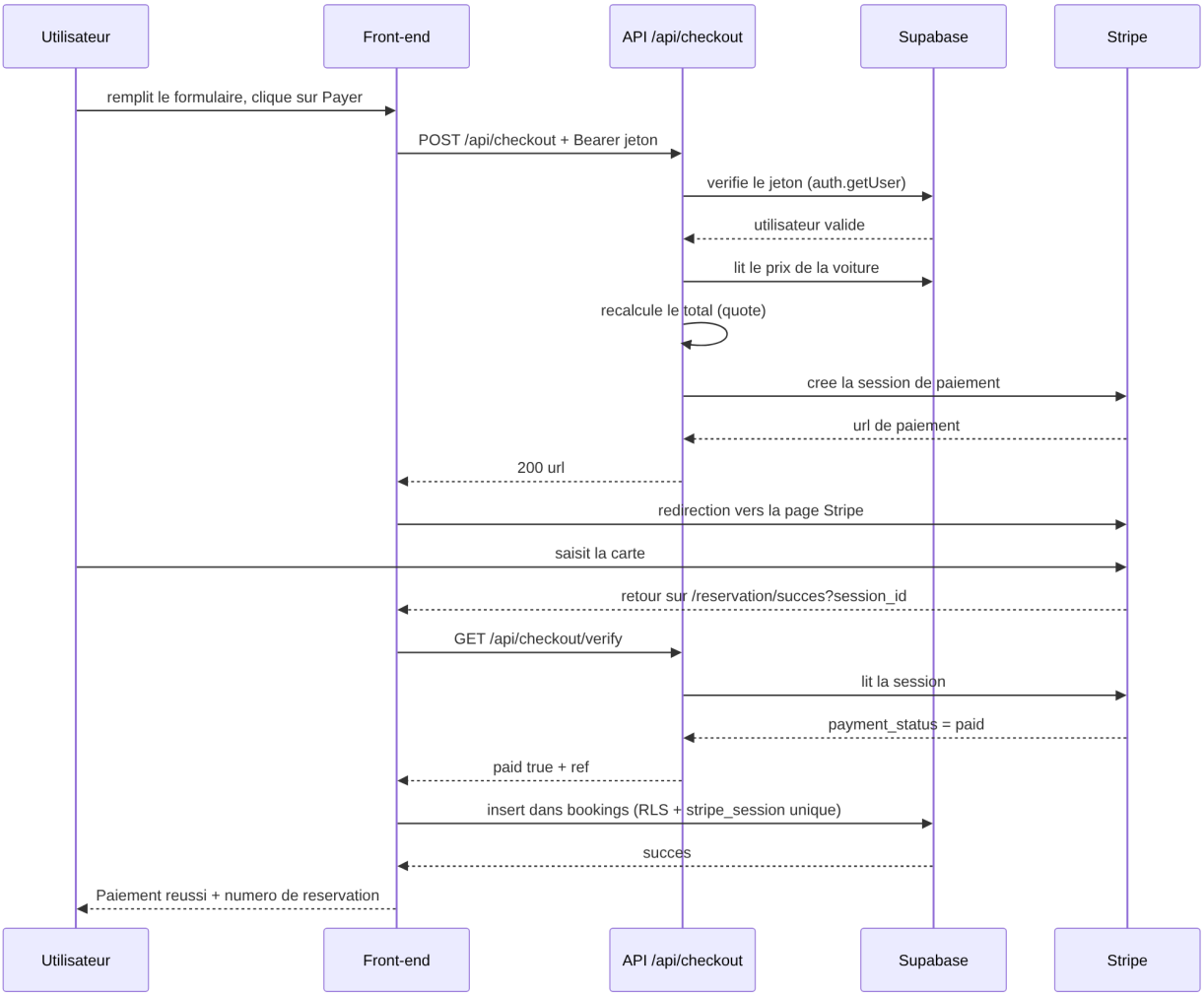
Fichier	Rôle
`cars.ts`	Types et données de secours (12 voitures, 6 agences)
`serverCatalog.ts`	Lecture du catalogue côté serveur, avec repli sur `cars.ts`
`catalog.tsx`, `catalogMap.ts`	Contexte React et conversion lignes de base ⇄ objets
`catalogAdmin.ts`	Écritures admin (enregistrer, supprimer, importer, liste des clients)
`pricing.ts`	Calcul du prix, remises (5 %, 10 %, 15 %) et options
`auth.ts`, `admin.ts`	Utilisateur connecté, rôle admin, toutes les réservations
`bookings.ts`	Lire, créer et annuler une réservation
`supabase.ts`	Client Supabase

4. High-Level Sequence Diagrams

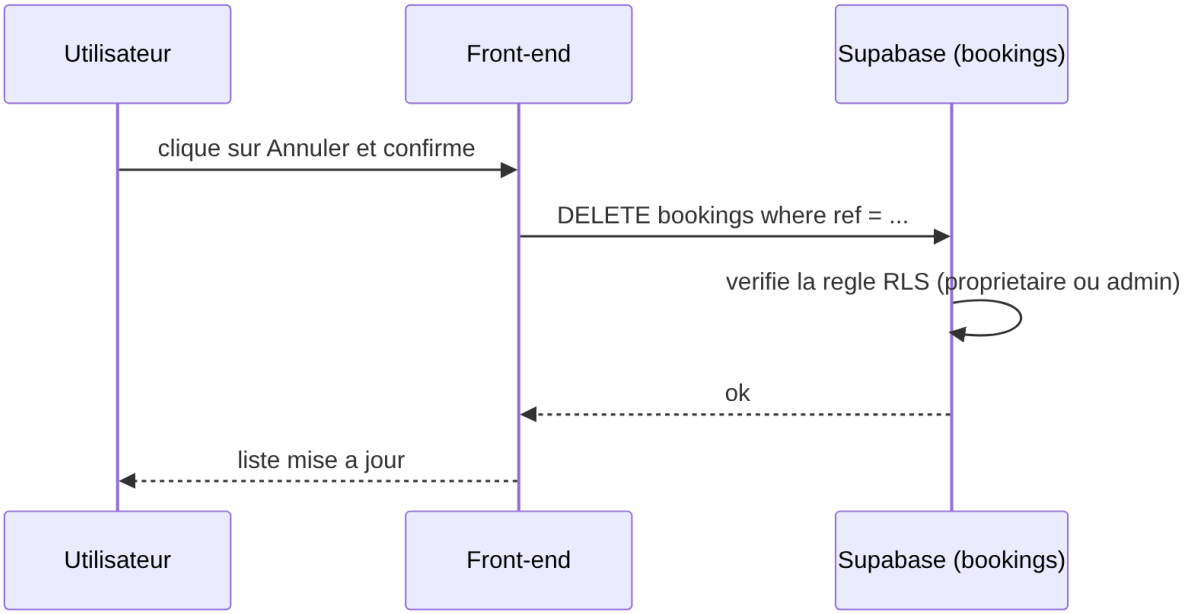
Cas 1 — Inscription / connexion obligatoire avant de réserver



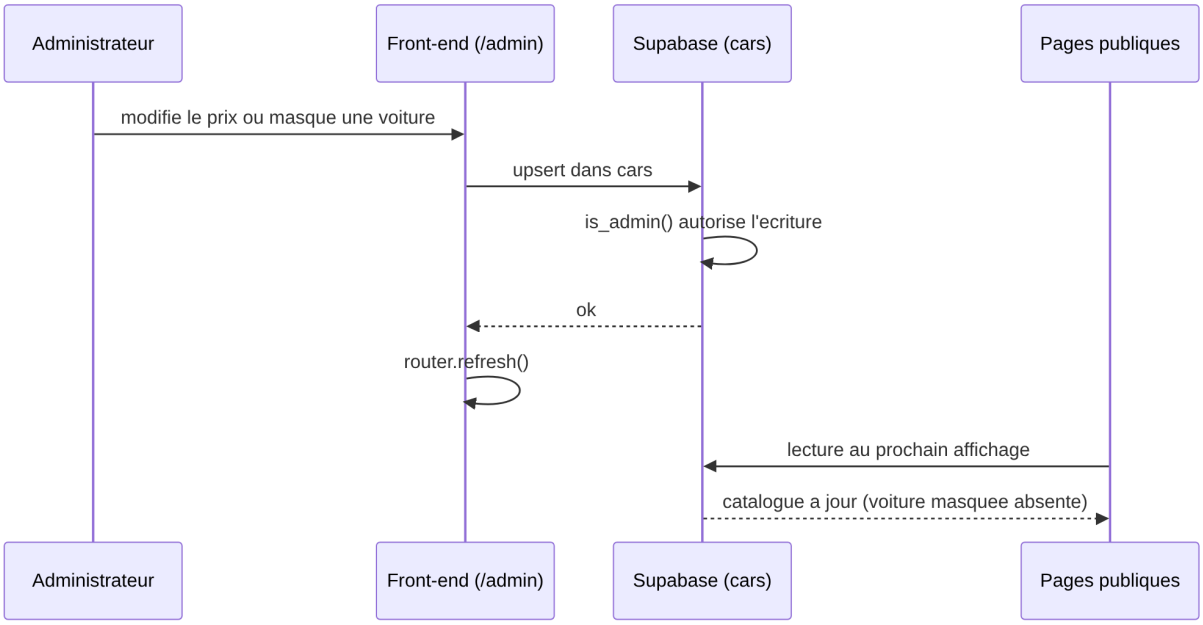
Cas 2 — Réservation avec paiement Stripe



Cas 3 — Annulation d'une réservation



Cas 4 — Un administrateur modifie une voiture



---

5. API Specifications

Driveo utilise trois familles d'interfaces : **(A)** deux routes API internes écrites dans le projet, **(B)** l'API de Supabase (authentification et données, appelée avec supabase-js), **(C)** l'API de Stripe. Toutes les communications passent en HTTPS et échangent du JSON.

A. API internes (routes Next.js)

Résumé

Méthode	Chemin	Rôle	Authentification
`POST`	`/api/checkout`	Crée une session de paiement Stripe	Obligatoire (jeton Bearer Supabase)
`GET`	`/api/checkout/verify`	Indique si une session Stripe est payée	Non (l'identifiant de session sert de preuve)

`POST /api/checkout`

Crée une session Stripe Checkout. Le prix est recalculé côté serveur à partir de la base de données : le montant envoyé par le navigateur n'existe pas, donc personne ne peut payer moins en modifiant la page.

En-têtes d'entrée

En-tête	Valeur	Obligatoire
`Content-Type`	`application/json`	Oui
`Authorization`	`Bearer <access_token Supabase>`	Oui

Corps JSON d'entrée

Champ	Type	Obligatoire	Règle de validation
`carId`	string	Oui	La voiture doit exister et être visible (`active`)
`du`	string `YYYY-MM-DD`	Oui	Date de début
`au`	string `YYYY-MM-DD`	Oui	Doit être après `du` (au moins 1 jour)
`opts`	string[]	Non	Identifiants d'options (`insurance` , `driver` , `gps` , `seat` , `wifi`) ; les inconnus sont ignorés
`ref`	string	Oui	Numéro de réservation généré par le front, renvoyé par Stripe
`email`	string	Non	Pré-remplit l'e-mail sur la page Stripe
`back`	string	Non	Paramètres d'URL pour revenir à la réservation si le client annule

Exemple de requête

```
POST /api/checkout
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{ "carId": "lamborghini-urus-performante", "du": "2026-11-04", "au": "2026-11-07",
  "opts": ["gps"], "ref": "DRV-48213", "email": "client@mail.com", "back": "car=lamborghini-urus-performante&du=2026-11-04&au=2026-11-07" }
```

Réponses (sortie)

Code	Quand	Corps JSON
`200`	Session créée	`{ "url": "https://checkout.stripe.com/c/pay/..." }`
`400`	Voiture inconnue ou masquée, dates invalides	`{ "error": "Reservation invalide." }`
`401`	Jeton absent ou invalide	`{ "error": "Compte requis : connecte-toi pour reserver." }`
`500`	Clé Stripe absente ou erreur Stripe	`{ "error": "Paiement non configure (STRIPE_SECRET_KEY manquante)." }` ou le message Stripe

Effets : crée une session Stripe en `mode: "payment"`, monnaie `eur`, montant en centimes ; `metadata.ref` = `ref` ; `success_url` = `/reservation/succes?session_id={CHECKOUT_SESSION_ID}` ; `cancel_url` = `/reservation?<back>`.

`GET /api/checkout/verify`

Demande à Stripe si la session a été payée. Le front n'enregistre la réservation que si `paid` vaut `true`.

Élément	Détail
Paramètre de requête	`session_id` (string, obligatoire)
Exemple	`GET /api/checkout/verify?session_id=cs_test_a1B2...`

Code	Corps JSON
`200`	`{ "paid": true, "ref": "DRV-48213", "amount": 2536 }` (`paid` vaut `false` si le paiement n'est pas terminé)
`400`	`{ "paid": false, "error": "Parametres manquants." }`
`500`	`{ "paid": false, "error": "<message Stripe>" }`

Format d'erreur commun : toute réponse non réussie contient une clé `error` avec un message lisible, affiché tel quel à l'utilisateur dans un encadré rouge.

B. API Supabase utilisées par le front

Le front ne code pas ces appels à la main : supabase-js les traduit en requêtes HTTP vers Supabase. Chaque requête porte `apikey: <clé publique anon>` et, si l'utilisateur est connecté, `Authorization: Bearer <jeton>`. Les droits sont décidés par les règles RLS décrites en section 3.

Action	Appel supabase-js	Requête HTTP équivalente	Entrée	Sortie
Inscription	`auth.signUp`	`POST /auth/v1/signup`	`{ email, password, data: { full_name } }`	`{ user, session }` ou erreur
Connexion	`auth.signInWithPassword`	`POST /auth/v1/token?grant_type=password`	`{ email, password }`	`{ access_token, refresh_token, user }`
Déconnexion	`auth.signOut`	`POST /auth/v1/logout`	—	`204`
Lire le catalogue	`from("cars").select`	`GET /rest/v1/cars?order=position.asc`	—	Tableau JSON de voitures (public)
Lister les agences	`from("agencies").select`	`GET /rest/v1/agencies?order=position.asc`	—	Tableau JSON (public)
Mes réservations	`from("bookings").select`	`GET /rest/v1/bookings?order=created_at.desc`	— (jeton)	Mes réservations (toutes pour un admin)
Enregistrer une réservation	`from("bookings").insert`	`POST /rest/v1/bookings`	Voir dictionnaire de données	`201` ou erreur
Annuler	`from("bookings").delete().eq("ref", ref)`	`DELETE /rest/v1/bookings?ref=eq.DRV-48213`	Numéro	`204`
Savoir si je suis admin	`from("admins").select`	`GET /rest/v1/admins?user_id=eq.<uid>`	—	Une ligne ou vide
Ajouter / modifier voiture ou agence (admin)	`from("cars").upsert`	`POST /rest/v1/cars` avec `Prefer: resolution=merge-duplicates`	Objet voiture	`201` ou `403` si non admin
Supprimer voiture ou agence (admin)	`from("cars").delete().eq("id", id)`	`DELETE /rest/v1/cars?id=eq.<id>`	Identifiant	`204` ou `403`
Liste des clients (admin)	`rpc("admin_clients")`	`POST /rest/v1/rpc/admin_clients`	—	Tableau `{ user_id, email, full_name, created_at, last_sign_in_at, bookings, spent }`

Codes d'erreur : `400` données invalides, `401` pas de jeton ou jeton expiré, `403` refusé par RLS (exemple : un client qui tente d'écrire dans `cars`), `409` doublon (une même session Stripe ne peut créer qu'une réservation).

C. API externes

Service	Endpoint utilisé	Usage	Pourquoi ce choix
Stripe Checkout	`POST /v1/checkout/sessions` et `GET /v1/checkout/sessions/{id}` (via la bibliothèque `stripe`)	Créer puis vérifier un paiement	Page de paiement hébergée : les numéros de carte ne passent jamais par notre serveur
Supabase	Voir B	Comptes et données	Sécurité par RLS, pas de serveur à écrire
Wikimedia Commons	Images par URL	Photos des voitures	Licences libres, crédit affiché sur le site

6. SCM and QA Plans

6.1 Gestion de version (SCM)

Dépôt et outils : Git, hébergé sur GitHub (`Janice91/metricflow`, nom hérité de la première idée du projet). Le code de production est celui de la branche `main`.

Stratégie de branches

Branche	Rôle	Durée de vie
`main`	Version en production, toujours déployable	Permanente
`feat/<sujet>`	Nouvelle fonctionnalité (ex. `feat/admin-voitures`)	Jusqu'à la fusion
`fix/<sujet>`	Correction d'un bug	Jusqu'à la fusion
`docs/<sujet>`	Documentation (README, dossier)	Jusqu'à la fusion

Pratique actuelle et cible : jusqu'ici, le projet étant mené seul sous contrainte de temps, les commits sont faits directement sur `main`, chacun déployé automatiquement. Pour la suite, la cible est de passer par des branches courtes et des Pull Requests relues par un pair (ou en auto-relecture avec la liste de contrôle ci-dessous), puis fusion dans `main`.

Convention de commits : message court à l'impératif, préfixé par le type : `feat:`, `fix:`, `docs:`, `chore:`, `refactor:`. Exemples réels de l'historique : « Driveo luxe : thème noir et or, comptes Supabase, vraies photos », « README : diagramme d'architecture front/back », « Admin back-office (voitures, agences, clients, réservations) + paiement Stripe test + compte obligatoire ».

Liste de contrôle de Pull Request

1. La compilation TypeScript passe (`npx tsc --noEmit`).
2. Le build de production passe (`npm run build`).
3. Le parcours impacté a été testé à la main.
4. Aucun secret dans le code (`git grep -n "sk_test\|sk_live"` ne renvoie rien).
5. La documentation est mise à jour si le comportement change.

Gestion des secrets

Secret	Où il vit	Visible côté navigateur ?
`NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`	`.env.local` (non versionné) et variables Vercel	Oui, par conception : la clé anon est publique et la sécurité repose sur RLS
`STRIPE_SECRET_KEY`	`.env.local` et variables Vercel (type Secret)	Non, utilisée uniquement dans les routes API

Le fichier `.env.local` est ignoré par Git (règle `.env*` du `.gitignore`), ce qui est vérifié avec `git status` avant chaque commit. La règle adoptée pour les jetons d'accès GitHub : type « fine-grained », limité au dépôt, expiration courte, révocation après usage.

Livraison continue (CI/CD)

1. `git push` sur GitHub.
2. Vercel détecte le push, installe les dépendances, lance `next build` et déploie : environ 30 à 60 secondes.
3. Une branche autre que `main` produit une URL de prévisualisation ; `main` met à jour la production.
4. En cas de problème, retour arrière en un clic (Vercel : « Promote » de l'ancien déploiement) ou `git revert` du commit fautif.

Versions et base de données : une version `v1.0.0` sera étiquetée (tag Git) avant la démo. Les scripts SQL (tables, règles RLS, fonctions) sont exécutés dans l'éditeur SQL de Supabase ; ils sont écrits pour être rejouables (`create ... if not exists`, `drop policy if exists`) et la cible est de les ranger dans le dépôt, dans un dossier dédié, pour garder le schéma versionné avec le code.

6.2 Plan qualité (QA)

Objectifs de qualité

Critère	Objectif
Fiabilité	Le site reste utilisable même si la base est injoignable (repli sur les données du code)
Sécurité	Un client ne voit jamais les données d'un autre ; aucun paiement possible sans compte ni à un prix modifié
Exactitude des prix	Le total payé est toujours celui calculé par le serveur
Utilisabilité	Parcours complet réalisable en moins de 2 minutes, sur ordinateur et mobile
Maintenabilité	Types TypeScript stricts, composants réutilisables, build sans erreur

Stratégie de test (niveaux)

Niveau	Quoi	Outil	Quand
Analyse statique	Types et syntaxe	`tsc --noEmit`, ESLint (`npm run lint`)	Avant chaque commit
Build	Compilation de production de toutes les routes	`npm run build`	Avant chaque push
Fonctionnel manuel	Parcours client et admin de bout en bout	Navigateur, comptes de test	Après chaque fonctionnalité
Paiement	Cartes de test Stripe	Cartes `4242 4242 4242 4242` (succès) et `4000 0000 0000 0002` (refus)	À chaque changement du paiement
Sécurité	RLS, accès sans compte, accès admin	Deux comptes (client et admin), appels directs	Avant la démo
Production	Même parcours sur le site déployé	`metricflow-tan.vercel.app`	Après chaque déploiement

Cas de test

ID	Fonction	Étapes	Résultat attendu	Statut
TC-01	Catalogue	Ouvrir `/voitures`	12 voitures avec photo, prix et note	Prévu
TC-02	Filtres	Filtrer par catégorie SUV	Seuls les SUV s'affichent ; « Réinitialiser » restaure la liste	Prévu
TC-03	Prix	Choisir 3 jours + GPS	Total = jours × prix − 5 % + options	Prévu
TC-04	Compte obligatoire (page)	Non connecté, cliquer sur « Créer un compte pour réserver »	Arrivée sur l'inscription avec voiture et dates conservées	Réussi
TC-05	Compte obligatoire (serveur)	Appeler `POST /api/checkout` sans jeton	`401` « Compte requis »	Réussi (03/11/2026, site en ligne)
TC-06	Inscription	Créer un compte	Retour à la réservation, utilisateur connecté	Réussi
TC-07	Paiement succès (local)	Payer avec `4242…`	Page « Paiement réussi », réservation visible dans « Mes réservations »	Réussi
TC-08	Paiement succès (production)	Idem sur le site en ligne	Idem	Réussi
TC-09	Paiement refusé	Payer avec `4000 0000 0000 0002`	Refus affiché par Stripe, aucune réservation créée	Prévu
TC-10	Isolation des données	Client A consulte « Mes réservations »	Aucune réservation d'un autre client	Réussi (03/11/2026, site en ligne)
TC-11	Annulation	Annuler une réservation	Elle disparaît de la liste	Prévu
TC-12	Accès admin refusé / redirection	Compte client ouvre `/admin` ; puis un compte admin se connecte	« Accès refusé » pour le client ; l'admin arrive directement sur `/admin`	Réussi (03/11/2026, site en ligne)
TC-13	Page d'administration	Compte admin ouvre `/admin`	4 onglets (Réservations, Voitures, Agences, Clients), sans pied de page ni liens clients	Réussi
TC-14	Import du catalogue	Admin clique sur « Importer » (voitures, agences)	Listes remplies depuis la base	Prévu
TC-15	Masquer une voiture	Admin masque une voiture	Absente de `/voitures`, présente dans l'admin	Prévu
TC-16	Prix modifié	Admin change un prix puis un client paie	Le montant Stripe correspond au nouveau prix	Prévu
TC-17	Écriture interdite	Compte client (non admin) envoie un `PATCH` sur `cars` via l'API REST de Supabase avec son jeton	Aucune ligne modifiée : réponse `200` avec `[]` (la politique RLS filtre l'écriture). Contrôle : la même ligne reste lisible en `GET`	Réussi (03/11/2026, site en ligne)

ID	Fonction	Étapes	Résultat attendu	Statut
TC-18	Liste des clients	Admin ouvre l'onglet Clients	Comptes, réservations et total dépensé	Prévu
TC-19	Repli	Base vide	Le site affiche les données du code	Prévu
TC-20	Mobile	Parcours sur écran étroit	Une colonne, pas de défilement horizontal	Prévu
TC-21	Prix falsifié	Compte connecté envoie `POST /api/checkout` avec de faux champs `total`, `price`, `pricePerDay` et `unit_amount` à 1 €	Stripe affiche le vrai prix (590,00 € pour une Corvette C8, 1 jour) : les champs envoyés sont ignorés	Réussi (03/11/2026, site en ligne)

Bilan actuel : 10 cas réussis sur 21, dont tous les cas de sécurité prévus : TC-05 (appel sans compte), TC-10 (isolation des données), TC-12 (accès admin refusé), TC-17 (écriture interdite par RLS) et TC-21 (prix falsifié), exécutés le 03/11/2026 sur le site en ligne. Les 11 autres cas fonctionnels seront exécutés avant la démo. Le build de production et la compilation TypeScript passent sans erreur sur la version déployée.

Critères de sortie (Definition of Done) : compilation et build sans erreur, cas Must testés, aucun bug bloquant ouvert, test réalisé sur le site en ligne, documentation à jour.

Journal de bugs (exemples rencontrés)

ID	Problème	Cause	Correction
BUG-01	Le formulaire de connexion se rechargeait sans connecter (environnement de développement)	Next.js bloquait les requêtes de rechargement à chaud venant d'un autre domaine	Ajout de `allowedDevOrigins` dans `next.config.ts`
BUG-02	Erreur 500 au premier chargement	La première compilation dépassait le délai du serveur proxy	Attendre la fin de la compilation puis recharger
BUG-03	En production : « An error occurred with our connection to Stripe »	Caractère invisible collé à la fin de la clé `STRIPE_SECRET_KEY`	Suppression et recréation de la variable (bouton « copier »), puis redéploiement
BUG-04	La page d'administration affichait des éléments clients	Navigation et pied de page partagés avec le site public	Barre dédiée et pied de page masqué sur `/admin`

Limites connues et améliorations prévues : la réservation est enregistrée par le navigateur après vérification du paiement ; si le client ferme la page avant, le paiement existe sans réservation. Un **webhook Stripe** (réception serveur-à-serveur de l'événement de paiement) rendrait ce cas impossible. Autres pistes : tests automatisés (Playwright pour le parcours, tests unitaires du calcul de prix), gestion des disponibilités, e-mail de confirmation.

7. Technical Justifications

7.1 Choix de la pile technique

Besoin	Choix	Alternatives étudiées	Arguments techniques	Compromis et limites
Framework web	**Next.js 16**	React seul (Vite), Django, Flask	Routage par dossiers, rendu côté serveur (le catalogue arrive déjà rempli, utile pour le référencement et la vitesse perçue), **routes API dans le même projet** (utile pour le paiement) ; déploiement natif sur Vercel	Framework récent à apprendre ; certaines pages sont rendues à chaque requête (voir 7.3)
Interface	**React 19**	Vue, HTML/JS pur	Composants réutilisables (carte voiture, onglets admin), état local simple, écosystème large	—
Langage	**TypeScript**	JavaScript	Types partagés entre front, API et base (`Car`, `Booking`) : une faute de champ est détectée à la compilation ; sert de documentation	Écriture un peu plus longue
Style	**Tailwind CSS 4**	CSS à la main, Bootstrap	Classes utilitaires : thème noir et or cohérent et rapide à itérer, pas de CSS inutilisé en production	Classes longues dans le JSX
Authentification et données	**Supabase** (Auth + PostgreSQL + RLS)	Firebase, Express + PostgreSQL maison	PostgreSQL relationnel adapté aux réservations ; **sécurité par ligne (RLS) dans la base**, donc même un appel direct à l'API ne contourne pas les règles ; comptes, jetons JWT et sessions prêts à l'emploi ; offre gratuite suffisante ; moins de code serveur à maintenir donc moins de failles	Dépendance à un service externe ; projet gratuit mis en pause après inactivité
Paiement	**Stripe Checkout** (page hébergée)	Stripe Elements (formulaire intégré), PayPal	Les numéros de carte ne transitent jamais par notre serveur, ce qui limite fortement les obligations de conformité PCI ; 3-D Secure géré par Stripe ; mode test complet pour la démo	Page de paiement hors de notre design
Hébergement	**Vercel**	Netlify, serveur à gérer soi-même	Déploiement automatique à chaque push, URLs de prévisualisation, retour arrière en un clic, variables secrètes ; conçu pour Next.js	Fonctions serverless : démarrage à froid possible
Code source	**Git + GitHub**	GitLab	Historique, partage avec les pairs, déclenche le déploiement	—
Images	**Wikimedia Commons**	Photos achetées, banque d'images	Licences libres, crédit affiché, coût nul	Qualité et cadrage variables

7.2 Décisions d'architecture et de sécurité

Décision	Pourquoi
Prix recalculé côté serveur à partir de la base, jamais accepté depuis le navigateur	Un client ne peut pas payer moins en modifiant la requête ; un changement de prix en admin s'applique immédiatement
Compte exigé à trois niveaux (bouton, page de paiement, serveur qui vérifie le jeton)	La vérification côté navigateur seule est contournable ; la barrière serveur garantit la règle
RLS + fonction `is_admin()` plutôt qu'un contrôle uniquement dans le code	La règle vit au plus près des données ; un oubli dans une page ne crée pas de fuite
Clé anon publique, clé secrète Stripe serveur uniquement (jamais préfixée `NEXT_PUBLIC_`)	La clé anon n'est dangereuse que sans RLS ; la clé secrète ne doit jamais atteindre le navigateur
Fonctions `security definer` limitées (`is_admin`, `admin_clients`) avec test d'admin intégré	Permet de lire `auth.users` (impossible autrement) sans l'exposer au public
Catalogue en base avec repli sur le code	L'admin peut modifier sans redéployer, et le site ne casse jamais si la base est vide ou en panne
Liens logiques `car_id` / `pickup` / `dropoff` sans clés étrangères	Supprimer une voiture ou une agence n'efface pas l'historique des réservations
Index unique sur `stripe_session`	Une session de paiement ne peut créer qu'une réservation, même si la page de succès est rechargée
Masquer plutôt que supprimer (champ `active`)	Réversible, garde l'historique, évite les erreurs

7.3 Performance, coût et évolutivité

- **Performance** : le rendu serveur évite un écran vide au chargement ; les pages sont dynamiques (`force-dynamic`) pour que les modifications de l'admin soient visibles immédiatement. La fonction `cache()` de React évite de lire deux fois le catalogue pendant une même requête. Évolution possible : cache avec revalidation à la demande (ISR) si le trafic augmente.
- **Coût** : Supabase (offre gratuite), Vercel (offre Hobby), Stripe (mode test gratuit). Seul le passage au paiement réel entraînerait des frais (commission Stripe par transaction).
- **Évolutivité** : base relationnelle indexable, services gérés et mis à l'échelle par les fournisseurs ; les routes API sans état s'exécutent en parallèle.
- **Maintenabilité** : peu de code serveur (deux routes), types stricts, séparation claire entre logique (`lib/`), composants et pages.

7.4 Risques techniques et réponses

Risque	Réponse
Paiement fait mais réservation non enregistrée (fermeture de page)	Webhook Stripe prévu en évolution ; en attendant, le paiement est retrouvable dans Stripe via `metadata.ref`
Fuite d'une clé secrète	Clés dans les variables d'environnement, jamais dans le dépôt ; révocation immédiate et régénération si exposition
Projet Supabase en pause	Vérification avant la démo, reprise en un clic
Dépendance à des services tiers	Choix de services standards et documentés ; repli sur les données du code pour le catalogue
Réservations concurrentes sur les mêmes dates	Non gérées dans cette version (US-09) ; à traiter avec une contrainte d'exclusion de dates en base